

Lightning, Field-Agnostic Super-Efficient Polynomial Commitment Scheme

Abstract

Polynomial commitment schemes (PCS) are a fundamental building block of modern zkSNARKs. In this paper, we propose *Lightning*, a new coding-based PCS that achieves state-of-the-art prover efficiency. Our main technical contribution is a new linear code family, the *Lightning code*, which can be instantiated from any base code with constant relative distance. Compared to the base code, Lightning code significantly reduces encoding cost by trading off relative distance. We integrate Lightning code into the standard coding-based PCS framework of Liger and Brakedown. Experimental results show that Lightning PCS reduces prover commitment time by up to $2.3\times$ compared to the fastest prover configuration of Brakedown, at the cost of a $2.1\times$ increase in proof size. Overall, Lightning provides a practical mechanism for trading proof size for prover efficiency in coding-based PCS constructions.

CCS Concepts

• Do Not Use This Code → Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

. 2018. Lightning, Field-Agnostic Super-Efficient Polynomial Commitment Scheme. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) enable a prover to convince a verifier of the correctness of a computation without revealing any additional information. Their succinctness and efficient verification make them particularly attractive for various applications, including blockchain rollups [5, 23], decentralized identity [24], anonymous payment [29], and verifiable machine learning inference [22].

Modern zkSNARK constructions largely follow the now-standard Polynomial Commitment Scheme (PCS)+ Polynomial Interactive Oracle Proof (PIOP) paradigm, instantiated in widely used proof systems such as PLONK, Marlin, and HyperPlonk. A PCS enables the prover to commit to a μ -variate polynomial f defined over a field $\mathbb{F}$ using a short commitment. Given $\vec{x} \in \mathbb{F}^\mu$, the prover can

create a proof to convince the verifier that $f(\vec{x}) = y$ for some $y \in \mathbb{F}$. Among these components, the PCS is often the dominant cost for the prover.

Since the PCS often dominates prover time, improving the efficiency of PCS has long been a central research direction in the study of zkSNARKs [18, 34, 35]. Over the past decade, multiple PCS designs have been proposed. The concept and initial construction of univariate PCS (namely, $\mu = 1$) originated from the pioneering work KZG [20], based on the discrete logarithm assumption and pairing groups. PST [26] extended the PCS concept to multilinear polynomials ($\mu > 1$, the degree in each variable is at most one). However, KZG and PST require a trusted setup and incur expensive group operations. In contrast, many recent PCS constructions are based on error-correcting codes, including Brakedown [18], Orion [33], Virgo [35], BaseFold [34], and WHIR [4]. These schemes have attracted significant attention due to their concrete efficiency, transparent setup, and plausible post-quantum security.

1.1 Our Contribution

In this paper, we propose *Lightning*, a new PCS that achieves state-of-the-art prover efficiency and outperforms all existing constructions in terms of commitment cost. Our design follows the classical coding-based PCS framework of Liger [3] and Brakedown [18], which constructs PCS from linear error-correcting codes and Merkle trees. Within this framework, any linear code with constant relative distance can be generically compiled into a PCS, and the overall prover complexity is dominated by the encoding cost of the underlying code. This abstraction highlights encoding efficiency as the central bottleneck in improving prover performance.

To reduce this encoding bottleneck, we introduce a new linear code family, the *Lightning code*, which can be instantiated from any base code with constant relative distance. Lightning code achieves substantially lower encoding cost than its base code while preserving constant relative distance. At a high level, Lightning code avoids encoding the entire message using the base code, yet still guarantees constant relative distance.

Concretely, given a message $m \in \mathbb{F}_q^n$, Lightning code first applies an efficiently computable linear map $A : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^{\alpha n}$, for some $\alpha < 1$, producing a compressed linear sketch mA . It then encodes only this sketch using a large-distance base code C_D . The resulting Lightning codeword is systematic:

$$C_L(m) := m \parallel C_D(mA),$$

so that the expensive encoder C_D is applied to an input smaller by a factor of α .

The key technical requirement on the linear map A is *sparsity preservation*: it must retain nontrivial information about sparse messages. Concretely, for every nonzero m with support size at most βn , we require $mA \neq 0$. Under this condition, any nonzero message contributes codeword weight either directly through the first part m , or through the encoded component $C_D(mA)$. As a result, the Lightning code achieves linear minimum distance.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

	Prover Time	Proof Size	Verifier Time	Field Agnostic
Ligero (CCS'17) [3]	$O(N \log N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$\times$
Virgo (S&P'20) [35]	$O(N \log N)$	$O(\log^2 N)$	$O(\log^2 N)$	$\times$
Orion (CRYPTO'22) [33]	$O(N)$	$O(\log^2 N)$	$O(\log^2 N)$	$\times$
Brakedown (CRYPTO'23) [18]	$O(N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$\checkmark$
BaseFold (CRYPTO'24) [34]	$O(N \log N)$	$O(\log^2 N)$	$O(\log^2 N)$	$\checkmark$
EA (CRYPTO'24) [7]	$O(N \log N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$\checkmark$
WHIR (EUROCRYPT'25) [4]	$O(N \log N)$	$O(\log^2 N)$	$O(\log^2 N)$	$\times$
Ours	$O(N)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$\checkmark$

Table 1: Performance of coding-based multilinear PCS. N is the size of the polynomial.

To enable efficient computation of mA , we construct A as a randomly sampled low-density parity-check (LDPC) matrix, specifically, a row-regular sparse binary matrix. Such matrices admit extremely fast multiplication: computing mA requires only $O(n)$ field additions and no field multiplications. In contrast, the base code encoding involves extensive field multiplications. Combined with the reduced input size for the base code, this design ensures that encoding with the Lightning code is substantially faster than encoding with its base code.

We instantiate Lightning code using the Brakedown code as the base code C_D and integrate it into standard coding-based PCS constructions, obtaining the Lightning PCS. Compared to the fastest prover configuration of Brakedown [18], Lightning PCS reduces prover commitment time by up to 2.3 $\times$, at the cost of a 2.1 $\times$ increase in proof size due to its smaller relative distance. These results demonstrate that Lightning provides a practical mechanism for enabling coding-based PCS constructions to trade prover efficiency for proof size.

We summarize our contributions as follows:

- We propose Lightning, a field-agnostic multilinear polynomial commitment scheme that substantially reduces prover encoding cost. Lightning achieves linear-time encoding dominated by field additions, and attains significantly lower concrete encoding cost than prior coding-based PCS constructions.
- Our core technical contribution is the Lightning code, a new linear code family that achieves efficient encoding by applying the expensive base code only to a compressed linear sketch of the message, rather than the full message, while still guaranteeing constant relative distance. Our analysis demonstrates that this linear compression can be instantiated using a random row-regular binary LDPC matrix, which preserves nontrivial information about sparse messages with overwhelming probability.
- We implement Lightning within standard coding-based PCS frameworks and benchmark it against existing schemes, including Brakedown, Orion, Ligero, and EA. Our experimental results show that Lightning improves commitment time by up to 2.3 $\times$ compared to the fastest Brakedown configuration, at the cost of 2.1 $\times$ increase in proof size; Lightning also improves the commitment time by up to 3 $\times$ compared to Orion, and 14 $\times$ compared to EA.

1.2 Related Work

In recent years, there has been significant progress in constructing concretely efficient polynomial commitment schemes (PCS). Existing constructions are based on a variety of cryptographic assumptions and techniques, including bilinear maps [20, 21, 26], discrete logarithms [32], groups of unknown order [12], lattices [2], and error-correcting codes [3, 6]. Among these, coding-based constructions are particularly appealing due to their transparent setup, simple algebraic structure, and plausible post-quantum security. Our work focuses on coding-based PCS.

The error-correcting codes adopted by prior PCS schemes can be broadly categorized into three classes: (1) foldable codes [34], (2) Brakedown code [18], and (3) Expander-Accumulation (EA) code [10]. These classes exhibit different trade-offs between encoding complexity and relative distance, which in turn directly affect prover efficiency and proof size in the resulting PCS.

Foldable codes, including Reed–Solomon codes and randomized foldable codes [34], typically achieve the largest relative distance among these three classes, but incur $O(n \log n)$ encoding complexity. Many PCS constructions adopt foldable codes, including Ligero [3], FRI [6], Virgo [35], Basefold [34], and WHIR [4]. Due to this quasilinear encoding cost, these PCS schemes incur $O(N \log N)$ prover commitment time (in field operations), where N is the polynomial size. This overhead becomes a major bottleneck in large-scale settings.

The Brakedown code was introduced by Golovnev et al. [18] by generalizing Spielman’s binary code [30] to large fields. Its key advantage is strictly linear encoding complexity in the message size, making it particularly attractive for improving prover efficiency. By adopting the Brakedown code, Brakedown PCS was the first to achieve $O(N)$ prover time (in terms of field operations). Subsequent works focused on improving the proof size: Orion [33] reduced Brakedown’s proof size from $O(\sqrt{N})$ to $O(\log^2 N)$ by leveraging proof recursion, but required NTT-friendly fields. Brakingbase [25] removed this restriction, making the construction field-agnostic while retaining Orion’s proof size. However, these follow-up works do not improve prover time beyond Brakedown, leaving a gap in further optimizing the encoding cost.

The EA code [10] achieves a larger concrete relative distance than the Brakedown code, at the cost of $O(n \log n)$ encoding complexity. Block et al. [7] designed a PCS based on the EA code, yielding a different trade-off between prover time and proof size compared to prior constructions.

Table 2: Encoding cost comparison of different linear codes used in zkSNARKs. n denotes the length of the message to be encoded.

		Rate	Distance	Encoding cost		
				$\mathbb{F}_\times$	$\mathbb{F}_+$	Total
Reed-Solomon code	$n = 2^{16}$	0.5	0.5	$16n$	$32n$	$24n$
	$n = 2^{20}$	0.5	0.5	$20n$	$40n$	$30n$
Brakedown code [18]	Parameters from improved analysis in Haböck [19] Table 1	0.704	0.02	$9.9n$	$9.9n$	$12.4n$
		0.680	0.03	$11.7n$	$11.7n$	$14.6n$
		0.657	0.04	$12.8n$	$12.8n$	$16.0n$
		0.610	0.05	$13.7n$	$13.7n$	$17.1n$
		0.619	0.06	$15.8n$	$15.8n$	$19.7n$
		0.581	0.07	$18.3n$	$18.3n$	$22.8n$
	Optimal cost parameter	0.899	0.016	$9.8n$	$9.8n$	$12.3n$
Lightning code		0.781	0.003	$2.0n$	$12.0n$	$5.0n$
		0.720	0.004	$2.7n$	$11.7n$	$5.6n$
		0.672	0.005	$3.4n$	$12.4n$	$6.5n$
		0.628	0.006	$4.1n$	$13.1n$	$7.4n$
		0.587	0.007	$4.9n$	$13.9n$	$8.4n$
		0.549	0.008	$5.7n$	$14.7n$	$9.4n$
		0.513	0.009	$6.6n$	$15.6n$	$10.5n$
		0.479	0.010	$7.6n$	$16.6n$	$11.8n$
		0.442	0.011	$8.8n$	$16.8n$	$13.0n$

• The encoding cost is estimated in equivalent $\mathbb{F}_\times$ operations, assuming $1 \mathbb{F}_\times = 4 \mathbb{F}_+$.

• The optimal encoding-cost parameters for the Brakedown code are obtained via a grid search over the full parameter space, minimizing the encoding cost.

More recently, Blaze [11] introduced an efficient PCS based on Repeat-Accumulate-Accumulate (RAA) codes [16]. However, Blaze is limited to binary fields, while our PCS supports general finite fields.

2 Preliminary

2.1 Multilinear extension

Define the identity function $eq(x, y)$, which takes two bit-strings x and y of the same length l as input. The function $eq(\cdot, \cdot)$ returns 1 if $x = y$; and 0 otherwise. Suppose $\tilde{eq}(\cdot, \cdot)$ is the multilinear extension of $eq(\cdot, \cdot)$. $\tilde{eq}(x, y) = \prod_{i=1}^l ((1 - x_i)(1 - y_i) + x_i y_i)$. We present the definition of Multilinear Extension as follows:

Definition 1 (Multilinear Extension [14]). Let $V : \{0, 1\}^\ell \rightarrow \mathbb{F}$ be a function. The *multilinear extension* of V is the unique polynomial $\tilde{V} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ such that $\tilde{V}(x_1, x_2, \dots, x_\ell) = V(x_1, x_2, \dots, x_\ell)$ for all $x_1, x_2, \dots, x_\ell \in \{0, 1\}^\ell$. $\tilde{V}$ can be expressed as $\tilde{V}(x_1, x_2, \dots, x_\ell) = \sum_{b \in \{0, 1\}^\ell} \tilde{eq}(x, b) \cdot V(b)$.

For a vector Z of length 2^ℓ , its multilinear extension $\tilde{Z}$ is defined as: $\tilde{Z}(x_1, \dots, x_\ell) = \sum_{i \in [2^\ell]} \tilde{eq}(x, \text{bin}(i - 1)) \cdot Z_i$.

2.2 Polynomial commitment

A polynomial commitment consists of three algorithms:

- $\text{PC.Commit}(\phi(\cdot))$: the algorithm outputs a commitment C of the polynomial $\phi(\cdot)$.
- $\text{PC.Prove}(\phi, \vec{x})$: given an evaluation point $\vec{x}$, the algorithm outputs a tuple $(\vec{x}, \phi(\vec{x}), \pi)$, where π is the proof.

- $\text{PC.VerifyEval}(\pi, \vec{x}, y, C)$: given $\pi, \vec{x}, y, C$, the algorithm checks if y is the correct evaluation. The algorithm outputs accept or reject.

Definition 2 ((Multivariate) Polynomial commitment scheme). A polynomial commitment scheme has the following properties:

- **Correctness.** For every polynomial ϕ and evaluation point $\vec{x}$, the following probability holds:

$$\Pr \left(\begin{array}{c} \text{PC.Commit}(\phi) \rightarrow C \\ \text{PC.Prove}(\phi, \vec{x}) \rightarrow \vec{x}, y, \pi \\ y = \phi(\vec{x}) \\ \text{PC.VerifyEval}(\pi, \vec{x}, y, C) \rightarrow \text{accept} \end{array} \right) = 1$$

- **Knowledge Soundness.** For any PPT adversary $\mathcal{P}^*$ with PC.Commit^* , PC.Prove^* , there exists a PPT extractor $\mathcal{E}$ such that the probability below is negligible:

$$\Pr \left(\begin{array}{c} \text{PC.Commit}^*(\phi^*) \rightarrow C^* \\ \text{PC.Prove}^*(\phi^*, \vec{x}) \rightarrow \vec{x}, y^*, \pi^* : \phi^* \leftarrow \mathcal{E}(C^*, \vec{x}, \pi^*, y^*) \wedge y^* \neq \phi^*(\vec{x}) \\ \text{PC.VerifyEval}(\pi^*, \vec{x}, y^*, C^*) \rightarrow \text{accept} \end{array} \right)$$

2.3 Error correction code

Definition 3 (Linear Code). A linear error-correcting code with message length k and codeword length n is a linear subspace $C \in \mathbb{F}^n$, such that there exists an injective mapping from message to codeword $E_C : \mathbb{F}^k \rightarrow C$, which is called the encoder of the code. Any linear combination of codewords is also a codeword. The rate of

the code is defined as $\frac{k}{n}$. The distance between two codewords u, v is the hamming distance denoted as $\Delta(u, v)$. The minimum distance is $d = \min_{u,v} \Delta(u, v)$. Such a code is denoted as $[n, k, d]$ linear code, and we further define its relative distance as $\text{rel_dist}(C) := \frac{d}{n}$.

2.4 Merkle Tree

Let $H : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ be a hash function. A Merkle Tree is a data structure that allows one to commit to $l = 2^d$ messages by a single hash value h , such that revealing any bit of the message require $d + 1$ hash values.

A Merkle hash tree is represented by a binary tree of depth d where l messages elements $m_1, m_2, \dots, m_l$ are assigned to the leaves of the tree. The values assigned to internal nodes are computed by hashing the value of its two child nodes. To reveal m_i , we need to reveal m_i together with the values on the path from m_i to the root. We denote the algorithm as follows:

- (1) $h \leftarrow \text{Merkle.Commit}(m_1, \dots, m_l)$.
- (2) $(m_i, \pi_i) \leftarrow \text{Merkle.Open}(m, i)$.
- (3) $\{\text{accept, reject}\} \leftarrow \text{Merkle.Verify}(\pi_i, m_i, h)$.

2.5 Building PCS from Error correction code and tensor IOP

We next illustrate the paradigm in [3, 7, 9, 18, 33] that builds a PCS on top of a linear code and tensor queries.

Brakedown [18] observed that a polynomial evaluation can be expressed as a tensor product. Here we only consider multilinear polynomial commitments, but the paradigm can be simply extended to univariate polynomials.

In particular, given a multilinear polynomial ϕ , its evaluation on input vector $x_0, x_1, \dots, x_{\log N - 1}$ is:

$$\phi(x_0, \dots, x_{\log N - 1}) = \sum_{i_0 \in \{0,1\}} \dots \sum_{i_{\log N - 1} \in \{0,1\}} w_{i_0 i_1 \dots i_{\log N - 1}} x_0^{i_0} \dots x_{\log N - 1}^{i_{\log N - 1}}$$

The degree of each variable is either 0 or 1 by the definition of a multilinear polynomial, and thus there are N monomials and coefficients with $\log N$ variables. Let $i_0 i_1 \dots i_{\log N - 1}$ be the binary representation of number i , i.e., $i = \sum_{j=0}^{\log N - 1} 2^j i_j$. We use w to denote the coefficients where $w[i] = w_{i_0 i_1 \dots i_{\log N - 1}}$. Similarly, we define $X_i = x_0^{i_0} x_1^{i_1} \dots x_{\log N - 1}^{i_{\log N - 1}}$. For $k = \sqrt{N}$, define vectors r_0, r_1 as $r_0 = \{X_0, X_1, \dots, X_{k-1}\}$, $r_1 = \{X_{0 \times k}, X_{1 \times k}, \dots, X_{(k-1) \times k}\}$. Then we have $X = r_0 \otimes r_1$. The polynomial evaluation is reduced to a tensor product:

$$\phi(x_0, x_1, \dots, x_{\log N - 1}) = \langle w, r_0 \otimes r_1 \rangle \quad (1)$$

Using the tensor IOP protocol in [9], one can build a polynomial commitment and we present the protocol in Protocol 1 for completeness. Here we reuse the notation k as it is exactly the message length of the linear code.

As shown in the protocol, to commit to a polynomial, PC.Commit parses the coefficients w as a $k \times k$ matrix W and encodes each row of it, resulting in a $k \times n$ matrix W' . Then the algorithm constructs a merkle tree commitment for every column of W' , and finally builds another Merkle tree on top of their roots as the final commitment.

To answer the tensor query, there are two checks in the protocol: a proximity check and a consistency check. The proximity check ensures that the matrix in the commitment is indeed close to a codeword. The consistency check ensures that $y = \langle w, r_0 \otimes r_1 \rangle$ assuming C is a commitment of a codeword.

Proximity check. The proximity check has two steps. First, the verifier sends a random vector γ to the prover, and the prover computes the linear combination of all rows of W with γ . Because of the property of a linear code, c_γ is a codeword. Second, the prover shows that c_γ is indeed computed from the committed tensor codeword. To do so, the verifier randomly selects t columns and the prover opens them with their Merkle tree proofs. The verifier checks that the inner product between each column and the random vector γ is equal to the corresponding element of c_γ . As shown in [9], if the linear code has a constant relative distance, the committed matrix is close to a tensor codeword with overwhelming probability.

Consistency check. The consistency check follows exactly the same steps of the proximity check. Instead of using a random vector from the verifier, the linear combination is done with r_0 of the tensor query $r_0 \otimes r_1$. Similarly, c_1 is a codeword of the linear code with message y_1 , and $\phi(x) = \langle y_1, r_1 \rangle$ by the definition of tensor product and polynomial evaluation. As shown in [9], by the check, if the committed matrix is close to a tensor codeword, then $y = \phi(x)$ with overwhelming probability. In particular, there exist an extractor to extract a polynomial ϕ from the commitment such that $y = \phi(x)$.

THEOREM 4. *Protocol 1 is a polynomial commitment scheme with soundness error $\frac{N}{P} + (1 - \frac{\delta}{3})^t + (1 - \frac{2\delta}{3})^t$, where δ is the relative distance of the code, and t is the number of opened columns in Protocol 1.*

With a linear code with constant relative distance of δ , by setting $t = \lambda \cdot \frac{1}{-\log_2(1 - \delta/3)}$, the soundness error can be made $2^{-\lambda}$. The proof size is $O(tk) = O(t\sqrt{N})$. By optimizing the dimensions of the coefficient matrix, the proof size can be improved to $O(\sqrt{tN})$ (see [18] for details).

Ligero [3], Brakedown [18], Orion [33], and EA [7] all adhere to the construction recipe described above, differing primarily in their choice of underlying linear code. Ligero relies on Reed–Solomon codes, Brakedown and Orion adopt the Brakedown code [18], while EA employs Expander-Accumulation codes [10]. These distinct code choices yield different trade-offs between prover time and proof size across the resulting PCS constructions.

2.6 LDPC Matrix

Definition 5. Let $A \in \{0, 1\}^{n \times m}$ be a binary matrix. We call A a low-density parity-check (LDPC) matrix [17] if the number of non-zero entries in A is much smaller than $n \cdot m$. Formally, $\frac{\text{weight}(A)}{n \cdot m} \ll 1$, where $\text{weight}(A) = |\{(i, j) : A_{i,j} = 1\}|$ is the number of ones in A .

LDPC matrices are an important tool in coding theory and cryptography [13, 15]. In this paper, we focus on *row-regular* LDPC matrices. In particular, we consider a uniformly random D -row-regular binary matrix $A \in \{0, 1\}^{n \times m}$, where each row contains exactly D non-zero entries. Each row of matrix A is sampled independently as a uniformly random binary vector of Hamming weight exactly D . We refer to such a distribution as the *random D -row-regular LDPC ensemble*.

Algorithm 1 The protocol for proving twisted evaluation. [7, 18, 33]

Public input: The evaluation point $\mathbf{x}$, transformed into a tensor product $r_0 \otimes r_1$ in Equation 1;
Private input: The polynomial ϕ , the coefficients of ϕ are denoted by w .
Let $E_C : \mathbb{F}^k \rightarrow \mathbb{F}^n$ be the encoding function of a $[n, k, d]$ -linear code. $N = k \times k$, if N is not a perfect square, we can pad it to the next perfect square. We use a python style notation to select the i -th column of a matrix $\text{mat}[:, i]$.

- 1: **function** PC.Commit(ϕ)
- 2: Parse the coefficients w as a $k \times k$ matrix W . The prover computes the encoding W' locally. In particular, W' is a $k \times n$ matrix by encoding each row of the matrix W using E_C .
- 3: **for** $i \in [n]$ **do**
- 4: Compute each Merkle tree leaf $\text{Root}_i = \text{Merkle.Commit}(W'[:, i])$.
- 5: Compute the Merkle tree root $\text{Root} = \text{Merkle.Commit}([\text{Root}_0, \dots, \text{Root}_{n-1}])$ and output Root as the commitment.
- 6: **function** PC.Open($\phi, \mathbf{x}$)
- 7: $\mathcal{P}$ receives a random vector $\gamma \in \mathbb{F}^k$ from $\mathcal{V}$, and computes $y_\gamma = \sum_{i=0}^{k-1} \gamma[i] W[i]$, $y_1 = \sum_{i=0}^{k-1} r_0[i] W[i]$.
- 8: $\mathcal{P}$ sends y_γ, y_1 to $\mathcal{V}$. $\mathcal{V}$ randomly samples t indices from $[n]$ as an array I and sends it to $\mathcal{P}$.
- 9: **for** $i \in I$ **do**
- 10: $\mathcal{P}$ sends $W'[:, i]$ and $(\text{Root}_i, \pi_i) \leftarrow \text{Merkle.Open}([\text{Root}_0, \dots, \text{Root}_{n-1}], i)$ to $\mathcal{V}$.
- 11: **function** PC.Verify($\pi, \mathbf{x}, y = \phi(\mathbf{x}), \text{Root}$)
- 12: Compute $c_\gamma = E_C(y_\gamma)$ and $c_1 = E_C(y_1)$.
- 13: **for** $\forall i \in I$ **do**
- 14: Check if $c_\gamma[i] == \langle \gamma, W'[:, i] \rangle$.
- 15: Check if $c_1[i] == \langle r_0, W'[:, i] \rangle$.
- 16: Check if $\text{Root}_i == \text{Merkle.Commit}(W'[:, i])$
- 17: Run $\text{Merkle.Verify}(\pi_i, \text{Root}_i, \text{Root})$.
- 18: Check if $y == \langle y_1, r_1 \rangle$. Output accept if all if the above conditions hold, otherwise, output reject.

3 Technical Overview

A large class of efficient polynomial commitment schemes (PCS), including Brakedown [18], Orion [33], Ligerio [3], and EA [7], follow a common paradigm: they build commitments and proofs from linear error-correcting codes combined with tensor IOPs. In these schemes, the dominant prover cost arises from repeatedly encoding rows of large witness matrices. Consequently, the encoding efficiency of the underlying code largely determines the overall prover time of the PCS.

High-level idea. Lightning accelerates error-correction code encoding by applying the base code on only a *compressed linear sketch* of the message, rather than the full message itself. Concretely, we first apply an efficiently computable linear map that reduces the message length to αn for some $\alpha < 1$. We then encode this compressed representation using the base code and concatenate it with the original message in a systematic manner. This design reduces the computational cost of the base encoding step by a factor of α ,

substituting it with a lightweight linear map, while still preserving linear distance guarantees.

From a base code to the Lightning code. Let C_D be a linear code over $\mathbb{F}_q$ with rate ρ and relative distance δ . Given a message $x \in \mathbb{F}_q^n$, Lightning applies a linear mapping $A : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^{\alpha n}$ and defines the Lightning codeword as

$$C_L(x) := x \parallel C_D(xA),$$

where “ $\parallel$ ” denotes concatenation. As long as the linear transform A is inexpensive to compute, encoding $C_L(x)$ is substantially cheaper than encoding $C_D(x)$, since the base code is applied only to a message of length αn .

Sparsity preservation. The key requirement on A is that it preserves non-trivial information on sparse messages. Specifically, we require A to be β -*sparsity-preserving*: for all $x \in \mathbb{F}_q^n$ satisfying $0 < |x|_0 < \beta n$, we have $xA \neq 0$. As shown in Lemma 1, this property implies that for all $x \in \mathbb{F}_q^n \setminus \{0\}$,

$$\|C_L(x)\|_0 \geq \min\left\{\beta n, \frac{\alpha n \delta}{\rho}\right\},$$

and hence Lightning achieves constant relative distance

$$\text{rel_dist}(C_L) \geq \min\left\{\frac{\beta \rho}{\rho + \alpha}, \frac{\delta \alpha}{\rho + \alpha}\right\}$$

Designing the linear transform. To ensure that xA can be computed efficiently, a natural choice is to take A as a sparse binary matrix—specifically, a Low-Density Parity-Check (LDPC) matrix [17]. We show that if $A \in \{0, 1\}^{n \times \alpha n}$ is sampled uniformly from the D -row-regular LDPC ensemble, then with high probability, matrix A is β -sparsity-preserving for an explicit $\beta = f(\alpha, D)$.

The unique column property. To show that matrix A is β -sparsity-preserving, we establish a sufficient combinatorial condition on the binary matrix A , which we call the *unique column property*, in Sections 4.2 and 4.3. We say that A satisfies the unique column property with parameter β if for every subset of rows $S \subseteq [n]$ with $|S| \leq \beta n$, there exists a column $c \in [\alpha n]$ such that among the rows indexed by S , exactly one row contains a 1 in column c . This condition implies β -sparsity-preservation: since xA is a linear combination of the rows of A indexed by $S = \text{supp}(x)$, such a unique column guarantees that $(xA)_c = x_{r^*} \neq 0$ for the unique contributing row $r^* \in S$, and hence $xA \neq 0$.

Intuition: a balls-into-buckets view. To build intuition for why a random D -row-regular binary matrix A is likely to satisfy the unique column property, we view the αn columns as αn buckets. Each row in S places D balls (ones) into D *distinct* buckets chosen uniformly at random, so the total number of balls is $D|S|$. When $D|S| \ll \alpha n$, collisions are rare and most balls land in previously empty buckets. As a result, with high probability, at least one occupied bucket contains exactly one ball, corresponding to a column that is unique for S .

Efficiency Comparison. Having established that the Lightning code preserves constant relative distance, we now turn to its encoding cost. Since each row of A contains exactly D nonzero entries (all equal to 1), computing xA requires only Dn field additions and no field multiplications. In practice, D can be as small as 9. By contrast, encoding the base code in schemes like Brakedown requires significantly heavier operations—specifically, at least $13.1n$ field

multiplications. Moreover, the encoding cost of the base code C_D is reduced by a factor of α . As a result, the overall encoding of the Lightning code is substantially faster than that of its base code.

Our experimental results confirm that Lightning PCS achieves a significant improvement in commitment time over existing coding-based PCS constructions. This improvement is primarily driven by Lightning's reduced reliance on expensive field multiplications, compared with prior efficiently encodable linear codes such as Brakedown [18], Expander-Accumulation (EA) [7, 10], and Reed–Solomon codes (as used in Ligerio [3]). In particular, Brakedown and EA incur a large number of field multiplications due to repeated sparse matrix-vector multiplications with random field coefficients, while Reed–Solomon encoding relies on NTT and likewise involves extensive field multiplications.

4 Methodology

We first define the Lightning encoding procedure as a systematic code that appends a base-code encoding of a compressed linear sketch (Section 4.1). In Section 4.2, we prove that Lightning inherits its constant relative distance provided that A satisfies the *unique-column* property. We then instantiate A as a random D -row-regular (LDPC) matrix and asymptotically bound the probability that this property fails. In Section 4.3, we further derive a tighter and efficiently computable upper bound on the failure probability. Finally, in Section 4.4, we combine this bound with an explicit encoding-cost analysis to select concrete parameters (α, D) that minimize encoding cost while meeting a target relative distance and setup failure probability.

4.1 Lightning Encoding

Lightning code accelerates ECC encoding by applying the base code on only a *compressed linear sketch* of the message, rather than the full message itself. Concretely, we first apply an efficiently computable linear map that reduces the message dimension to αn for some $\alpha < 1$. We then encode this compressed representation using the base code and concatenate it with the original message in a systematic manner.

Let C_D be a linear code over $\mathbb{F}_q$ with rate ρ and relative distance δ . Given a message $x \in \mathbb{F}_q^n$, Lightning applies a linear mapping $A : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^{\alpha n}$ and defines the Lightning codeword as

$$C_L(x) := x \parallel C_D(xA),$$

where “ $\parallel$ ” denotes concatenation. As long as the linear transform A is inexpensive to compute, encoding $C_L(x)$ is substantially cheaper than encoding $C_D(x)$, since the base code is applied only to a message of length αn .

The matrix A used here should satisfy the following property.

Definition 6 (Sparsity Preservation). Matrix $A \in \mathbb{F}_q^{n \times \alpha n}$ is β -sparsity-preserving if for every nonzero vector $x \in \mathbb{F}_q^n$ with $0 < \|x\|_0 < \beta n$, it holds that $xA \neq 0$.

The following lemma shows that if A satisfies sparsity preservation, then the resulting Lightning code C_L achieves constant relative distance.

LEMMA 1. Let $A \in \mathbb{F}_q^{n \times \alpha n}$ be β -sparsity-preserving (Definition 6). Let C_D be any error-correcting code with rate ρ and relative distance

δ . Define the Lightning code $C_L : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^{n+\alpha n/\rho}$ by

$$C_L(x) = x \parallel C_D(xA)$$

Then for every $x \in \mathbb{F}_q^n \setminus \{0\}$,

$$\|C_L(x)\|_0 \geq \min \left\{ \beta, \frac{\alpha \delta}{\rho} \right\} \cdot n$$

Consequently, C_L has relative distance

$$\text{rel_dist}(C_L) \geq \min \left\{ \frac{\beta \rho}{\rho + \alpha}, \frac{\delta \alpha}{\rho + \alpha} \right\}$$

PROOF. Since C_D has rate ρ , the codeword $C_D(xA)$ has length $\alpha n/\rho$. We consider two cases.

Case 1: $\|x\|_0 < \beta n$. By β -sparsity preservation, we have $xA \neq 0$, and thus

$$\|C_D(xA)\|_0 \geq \delta \cdot \frac{\alpha n}{\rho}$$

Therefore,

$$\|C_L(x)\|_0 \geq \|C_D(xA)\|_0 \geq \frac{\alpha \delta}{\rho} \cdot n$$

Case 2: $\|x\|_0 \geq \beta n$. Then

$$\|C_L(x)\|_0 \geq \|x\|_0 \geq \beta n$$

Combining the two cases yields

$$\|C_L(x)\|_0 \geq \min \left\{ \beta, \frac{\alpha \delta}{\rho} \right\} \cdot n$$

Finally, since C_L has block length $n + \alpha n/\rho = n(\rho + \alpha)/\rho$, we obtain

$$\text{rel_dist}(C_L) = \min_{x \neq 0} \frac{\|C_L(x)\|_0}{n + \alpha n/\rho} \geq \min \left\{ \frac{\beta \rho}{\rho + \alpha}, \frac{\delta \alpha}{\rho + \alpha} \right\}$$

□

We now turn to constructing a matrix A that is β -sparsity-preserving while still allowing xA to be computed efficiently. As mentioned in Section 3, we instantiate A as a uniformly random D -row-regular binary LDPC matrix. Note that under this choice, computing xA requires only $O(nD)$ field additions.

The remaining task is to show that such a random matrix $A \in \{0, 1\}^{n \times \alpha n}$ satisfies the β -sparsity-preserving property with high probability, for an explicit parameter $\beta = f(\alpha, D)$.

4.2 Unique Column Property and Asymptotic Analysis of Failure Probability

To simplify the analysis, we prove a strictly stronger combinatorial condition, which we call the *unique column property*: for every row subset $S \subseteq [n]$ with $|S| \leq \beta n$, there exists a column of A that has exactly one nonzero entry among the rows in S .

The key intuition is that the product xA can be viewed as a linear combination of the rows of A , with coefficients given by the entries of x . Let $S = \text{supp}(x)$. Under the unique column property, S admits a *unique column* c , meaning that exactly one row $r^* \in S$ has a nonzero entry in column c . Consequently, the c -th coordinate of xA equals $x_{r^*} \neq 0$, implying $xA \neq 0$ and hence establishing β -sparsity preservation.

		1	2	3	4	5	
1	1	1					Unique columns:
2		1				1	$U(\{1,2\})=\{1,5\}$
3			1	1			$U(\{1,6\})=\{1,2,3,5\}$
4	1				1		$U(\{2,5,6\})=\{3,4\}$
5		1		1			$U(\{3,4,5\})=\{1,2,3\}$
6			1		1		

For any vector x , $U(\text{supp}(x)) \neq \emptyset \Rightarrow xA \neq 0$

Example:

$$x = [0, 2, 0, 0, 1, 1], \quad y = xA$$

$$\text{supp}([0, 2, 0, 0, 1, 1]) = \{2, 5, 6\}$$

$$U(\{2, 5, 6\}) = \{3, 4\} \Rightarrow y_3 \neq 0, y_4 \neq 0$$

Figure 1: For a 6×5 2-row-regular LDPC matrix A , we illustrate several instances of $U(S)$ for different choices of S . The existence of a unique column for set S guarantees that $xA \neq 0$ for any nonzero vector x with $\text{supp}(x) = S$.

Formally, for a row set $S \subseteq [n]$, we define the column weight

$$\text{wt}_S(c) := \sum_{r \in S} A_{r,c}.$$

A column c is called *unique* for S if $\text{wt}_S(c) = 1$, and we denote the set of such columns by

$$U(S) := \{c \in [an] : \text{wt}_S(c) = 1\}.$$

We have the following lemma:

LEMMA 2 (UNIQUE COLUMN $\Rightarrow \beta$ -SPARSITY-PRESERVING). *If for every $S \subseteq [n]$ with $|S| \leq \beta n$ we have $U(S) \neq \emptyset$, then for every prime q , every nonzero $x \in \mathbb{F}_q^n$ with $\|x\|_0 < \beta n$ we have $xA \neq 0$.*

PROOF. Let $S = \text{supp}(x)$. Pick $c \in U(S)$ and let $r^* \in S$ be the unique row with $A_{r^*,c} = 1$. Then $(xA)_c = \sum_{r \in S} x_r A_{r,c} = x_{r^*} \neq 0$, hence $xA \neq 0$. $\square$

Lemma 2 shows that the unique column property implies β -sparsity preservation. Therefore, it remains to show that $U(S) \neq \emptyset$ for all row subsets S with $|S| \leq \beta n$ with high probability.

We provide some intuition for why, for a uniformly random D -row-regular matrix $A \in \{0, 1\}^{n \times an}$, a small row subset S is very likely to satisfy $U(S) \neq \emptyset$. View the an columns as an buckets. Each row in S places D balls (ones) into D distinct buckets chosen uniformly at random, so the total number of balls is $D|S|$.

When $D|S| \ll an$, the process operates in a sparse regime where collisions are rare: most balls land in previously empty buckets. Consequently, the number of occupied buckets is close to $D|S|$, making it unlikely that every occupied bucket has load at least 2. Therefore, with overwhelming probability, there exists at least one bucket with load exactly 1, corresponding to a unique column in $U(S)$.

In the remainder of this section, we formally upper bound the failure probability

$$\Pr[\exists S \subseteq [n], |S| \leq \beta n, U(S) = \emptyset].$$

The following lemma upper bounds the failure probability conditioned on the row subset size s .

LEMMA 3. *Let $A \in \{0, 1\}^{n \times an}$ be a uniformly random D -row-regular binary matrix. For any integer $s \in [1, \frac{2an}{D}]$, we have*

$$\Pr\left[\exists S \subseteq [n], |S| = s, |U(S)| = \emptyset\right] \leq \left(\frac{Cs}{n}\right)^{s(\frac{D}{2}-1)},$$

where $C = e^{\frac{D+2}{D-2}} \left(\frac{D}{2\alpha}\right)^{\frac{D}{D-2}}$ is a constant.

PROOF. We first fix a set $S \subseteq \{1, \dots, n\}$ with $|S| = s$. And we upper bound the probability of “bad” event $|U(S)| = \emptyset$.

Define covered column set of S as $H_S = \{c \in \{1, \dots, an\}, \sum_{r \in S} A_{r,c} \geq 1\}$.

If $|U(S)| = \emptyset$ occurs, then for each column c , $\sum_{r \in S} A_{r,c} = 0$ or $\sum_{r \in S} A_{r,c} \geq 2$. Under this condition, by the definition of H_S , for $\forall c \in H_S$, $\sum_{r \in S} A_{r,c} \geq 2$. Thus, we have:

$$\sum_{r \in S} \sum_{c \in \{1, \dots, an\}} A_{r,c} \geq 2|H_S|$$

Since each row in A has D unique 1's, $\sum_{r \in S, c \in \{1, \dots, an\}} A_{r,c} = Ds$.

Thus, if $|U(S)| = \emptyset$ occurs, $|H_S| \leq \frac{Ds}{2}$. We next upper bound the probability of $|U(S)| = \emptyset$ by upper bounding the probability of $|H_S| \leq \frac{Ds}{2}$.

If $|H_S| \leq \frac{Ds}{2}$, then there exists a set $T \subseteq \{1, \dots, an\}$ of size $\frac{Ds}{2}$ such that $H_S \subseteq T$. For each row index r , define its set of nonzero columns as

$$\text{nz}(r) = \{c \in \{1, \dots, an\} : A_{r,c} = 1\}.$$

Therefore, we have the following implication:

$$\begin{aligned} |U(S)| = \emptyset &\implies |H_S| \leq \frac{Ds}{2} \\ &\implies \exists T \subseteq \{1, \dots, an\}, |T| = \frac{Ds}{2} \text{ such that } \forall r \in S, \text{nz}(r) \subseteq T. \end{aligned}$$

Hence,

$$\begin{aligned} \Pr[|U(S)| = \emptyset] &\leq \Pr\left[\exists T : |T| = \frac{Ds}{2} \wedge \forall r \in S, \text{nz}(r) \subseteq T\right] \\ &\leq \sum_{\substack{T \subseteq \{1, \dots, an\} \\ |T| = \frac{Ds}{2}}} \Pr\left[\forall r \in S, \text{nz}(r) \subseteq T\right], \end{aligned}$$

where the last inequality follows from the union bound.

Fix an arbitrary set $T \subseteq \{1, \dots, an\}$ with $|T| = \frac{Ds}{2}$. For a given r , the probability that $\text{nz}(r) \subseteq T$ is

$$\Pr[\text{nz}(r) \subseteq T] = \frac{\binom{\frac{Ds}{2}}{D}}{\binom{an}{D}} = \prod_{i=0}^{D-1} \frac{\frac{Ds}{2} - i}{an - i} \leq \left(\frac{\frac{Ds}{2}}{an}\right)^D.$$

Since the sampling of $\text{nz}(r)$ is independent for each $r \in S$, we obtain

$$\Pr\left[\forall r \in S, \text{nz}(r) \subseteq T\right] = \prod_{r \in S} \Pr[\text{nz}(r) \subseteq T] \leq \left(\frac{\frac{Ds}{2}}{an}\right)^{Ds}.$$

Finally, taking a union bound over all choices of T with $|T| = \frac{Ds}{2}$ gives

$$\Pr[|U(S)| = \emptyset] \leq \binom{an}{\frac{Ds}{2}} \left(\frac{\frac{Ds}{2}}{an}\right)^{Ds}.$$

We can use the Stirling formula to upper bound $\left(\frac{\alpha n}{\frac{Ds}{2}}\right)$ as:

$$\left(\frac{\alpha n}{\frac{Ds}{2}}\right) \leq \left(\frac{e\alpha n}{\frac{Ds}{2}}\right)^{\frac{Ds}{2}}$$

Then we obtain

$$\Pr[|U(S)| = \emptyset] \leq \left(\frac{e\alpha n}{\frac{Ds}{2}}\right)^{\frac{Ds}{2}} \left(\frac{\frac{Ds}{2}}{\alpha n}\right)^{Ds} \quad (2)$$

Next we apply a union bound over all S of size s . There are $\binom{n}{s}$ such sets, and $\binom{n}{s} \leq \left(\frac{en}{s}\right)^s$. Combining this with (2), we get

$$\begin{aligned} \Pr[\exists S \subseteq \{1, \dots, n\}, |S| = s, |U(S)| = \emptyset] &\leq \left(\frac{e\alpha n}{\frac{Ds}{2}}\right)^{\frac{Ds}{2}} \left(\frac{\frac{Ds}{2}}{\alpha n}\right)^{Ds} \left(\frac{en}{s}\right)^s \\ &= \left(\frac{eDs}{2\alpha n}\right)^{\frac{Ds}{2}} \left(\frac{en}{s}\right)^s \\ &= e^{\frac{Ds}{2} + s} \left(\frac{D}{2\alpha}\right)^{\frac{Ds}{2}} \left(\frac{s}{n}\right)^{\frac{Ds}{2} - s} \\ &= \left(e^{\frac{D}{2} + 1} \left(\frac{D}{2\alpha}\right)^{\frac{D}{2}} \left(\frac{s}{n}\right)^{\frac{D}{2} - 1}\right)^s \\ &= \left(e^{\frac{D+2}{2}} \left(\frac{D}{2\alpha}\right)^{\frac{D}{2} - 2} \frac{s}{n}\right)^{s(\frac{D}{2} - 1)} \end{aligned}$$

Let constant $C = e^{\frac{D+2}{2}} \left(\frac{D}{2\alpha}\right)^{\frac{D}{2} - 2}$. By the above inequality, we can derive that:

$$\Pr\left[\exists S \subseteq [n], |S| = s, |U(S)| = \emptyset\right] \leq \left(\frac{Cs}{n}\right)^{s(\frac{D}{2} - 1)} \quad (3)$$

which ends our proof. $\square$

Lemma 3 bounds the failure probability for a *fixed* subset size s . We now extend this bound to all subsets of size at most βn by taking a union bound over $s = 1, 2, \dots, \lfloor \beta n \rfloor$. By choosing $\beta = \frac{0.98n}{C}$, the resulting summation is dominated by a rapidly decaying geometric series, which yields an overall failure probability of $O(n^{3-D})$. This gives the following theorem.

THEOREM 7 (PROBABILITY OF NO UNIQUE COLUMN). $\forall$ integer $D \geq 4$, let $A \in \{0, 1\}^{n \times \alpha n}$ be a uniformly random D -row-regular binary matrix.

$$\Pr\left[\exists S \subseteq [n], |S| \leq \left\lfloor \frac{0.98n}{C} \right\rfloor, |U(S)| = \emptyset\right] \leq O(n^{3-D}),$$

where constant $C = e^{\frac{D+2}{2}} \left(\frac{D}{2\alpha}\right)^{\frac{D}{2} - 2}$.

PROOF. We upper bound the probability by a union bound over the set size s and then apply Lemma 3. Let $s_{\max} := \left\lfloor \frac{0.98n}{C} \right\rfloor$. Then

$$\begin{aligned} \Pr\left[\exists S \subseteq [n], |S| \leq s_{\max}, U(S) = \emptyset\right] &\leq \sum_{s=2}^{s_{\max}} \Pr\left[\exists S \subseteq [n], |S| = s, U(S) = \emptyset\right] \\ &\leq \sum_{s=2}^{s_{\max}} \left(\frac{Cs}{n}\right)^{s(\frac{D}{2} - 1)} \quad (4) \end{aligned}$$

Define

$$F(s) := \left(\frac{Cs}{n}\right)^{s(\frac{D}{2} - 1)}.$$

To control the sum in (4), we study the maximum of $F(s)$ on the interval $s \in [2, \frac{0.98n}{C}]$ (treating s as a real variable). Since

$$\ln F(s) = \left(\frac{D}{2} - 1\right) s \ln \left(\frac{Cs}{n}\right),$$

we have

$$\frac{d \ln F(s)}{ds} = \left(\frac{D}{2} - 1\right) \left(\ln \left(\frac{Cs}{n}\right) + 1\right).$$

Hence $\frac{d \ln F(s)}{ds} < 0$ for $s < \frac{n}{Ce}$ and $\frac{d \ln F(s)}{ds} > 0$ for $s > \frac{n}{Ce}$. Therefore $F(s)$ decreases on $[2, \frac{n}{Ce}]$ and increases on $[\frac{n}{Ce}, \frac{0.98n}{C}]$, so its maximum over $[2, \frac{0.98n}{C}]$ is attained at an endpoint:

$$\max_{2 \leq s \leq \frac{0.98n}{C}} F(s) \leq \max \left\{ F(2), F\left(\frac{0.98n}{C}\right) \right\}.$$

We compute these two values explicitly:

$$F(2) = \left(\frac{2C}{n}\right)^{2(\frac{D}{2} - 1)} = \left(\frac{2C}{n}\right)^{D-2},$$

and

$$F\left(\frac{0.98n}{C}\right) = (0.98)^{\left(\frac{D}{2} - 1\right) \cdot \frac{0.98n}{C}}.$$

Plugging into (4) and using $s_{\max} \leq \frac{0.98n}{C}$ gives

$$\begin{aligned} \Pr\left[\exists S \subseteq [n], |S| \leq s_{\max}, U(S) = \emptyset\right] &\leq s_{\max} \cdot \max_{2 \leq s \leq s_{\max}} F(s) \\ &\leq \max \left\{ \frac{0.98n}{C} \cdot \left(\frac{2C}{n}\right)^{D-2}, \frac{0.98n}{C} \cdot 0.98^{\left(\frac{D}{2} - 1\right) \cdot \frac{0.98n}{C}} \right\}. \end{aligned}$$

As $n \rightarrow \infty$ (for fixed α, D), the second term is $o(1/\text{poly}(n))$. For the first term, we have

$$\frac{n}{C} \left(\frac{2C}{n}\right)^{D-2} = (2C)^{D-2} C^{-1} n^{-(D-3)} = O(n^{3-D}),$$

where the hidden constant depends only on α and D . Combining the two terms proves the theorem. $\square$

Theorem 7 implies that for a uniformly random D -row-regular binary matrix A , with probability at least $1 - O(n^{3-D})$, every row subset $S \subseteq [n]$ of size at most $\left\lfloor \frac{0.98n}{C} \right\rfloor$ admits a unique column in A , i.e., $U(S) \neq \emptyset$.

4.3 Concrete and Computable Failure Probability Bounds

Section 4.2 shows that as $n \rightarrow \infty$ (with fixed α and $D \geq 4$), the failure probability becomes inverse-polynomially small. However, this asymptotic result is not directly informative for practical PCS instantiations, where concrete security guarantees are required. In this subsection, we derive a tighter and efficiently computable upper bound on the failure probability under concrete parameter choices.

Table 3: Sample parameters of Lightning code instantiated with the Brakedown base code. For each target relative distance, we report the minimum-cost parameters satisfying $\Pr[\text{failure}] \leq 2^{-100}$ at $n = 2^{20}$.

Rate	Distance	Encoding Cost		$\Pr[\text{failure}]$	α	D	β	ρ	δ
		$\mathbb{F}_\times$	$\mathbb{F}_+$						
0.781	0.003	$2.0n$	$12.0n$	$2^{-105.9}$	0.198	10	0.0039	0.704	0.02
0.720	0.004	$2.7n$	$11.7n$	$2^{-105.7}$	0.274	9	0.0056	0.704	0.02
0.672	0.005	$3.4n$	$12.4n$	$2^{-108.6}$	0.343	9	0.0074	0.704	0.02
0.628	0.006	$4.1n$	$13.1n$	$2^{-111.2}$	0.417	9	0.0096	0.704	0.02
0.587	0.007	$4.9n$	$13.9n$	$2^{-113.4}$	0.495	9	0.0119	0.704	0.02
0.549	0.008	$5.7n$	$14.7n$	$2^{-115.4}$	0.579	9	0.0146	0.704	0.02
0.513	0.009	$6.6n$	$15.6n$	$2^{-117.3}$	0.669	9	0.0176	0.704	0.02
0.479	0.010	$7.6n$	$16.6n$	$2^{-119.1}$	0.766	9	0.0209	0.704	0.02
0.442	0.011	$8.8n$	$16.8n$	$2^{-104.3}$	0.888	8	0.0249	0.704	0.02

Following the analysis in Theorem 7, we bound the failure probability by separately considering row sets of different sizes:

$$\Pr \left[\exists S \subseteq [n], 2 \leq |S| \leq \left\lfloor \frac{0.98n}{C} \right\rfloor, U(S) = \emptyset \right] \\ \leq \sum_{s=2}^{\left\lfloor \frac{0.98n}{C} \right\rfloor} \Pr \left[\exists S \subseteq [n], |S| = s, U(S) = \emptyset \right]$$

The proof of Theorem 7 further upper-bounds the right-hand side by $\left\lfloor \frac{0.98n}{C} \right\rfloor \cdot F(2)$, where $F(s) = \left(\frac{Cs}{n} \right)^{s(\frac{D}{2}-1)}$. While this bound is efficiently computable, it is rather loose for concrete parameters. We therefore refine the analysis to obtain a tighter, yet still efficiently computable, upper bound, stated below.

THEOREM 8. *For any integer $D \geq 4$, let $A \in \{0,1\}^{n \times \alpha n}$ be a uniformly random D -row-regular binary matrix.*

$$\sum_{s=2}^{\left\lfloor \frac{0.98n}{C} \right\rfloor} \Pr \left[\exists S \subseteq [n], |S| = s, U(S) = \emptyset \right] \\ \leq \frac{n(n-1)}{2^{\binom{\alpha n}{D}}} + 1.8 \left(\frac{3C}{n} \right)^{3(\frac{D}{2}-1)} + \frac{n}{C} \cdot 0.98^{\left\lceil \frac{n}{1.5eC} \right\rceil (\frac{D}{2}-1)}$$

where the constant C is defined as $C = e^{\frac{D+2}{D-2}} \left(\frac{D}{2\alpha} \right)^{\frac{D}{D-2}}$.

PROOF. For the first term of the sum, $\Pr \left[\exists S \subseteq [n], |S| = 2, U(S) = \emptyset \right]$, according to the analysis in Theorem 7, it is the dominating term of the sum under typical parameters.

Our key observation is that, we can compute its exact value precisely, instead of applying a series of relaxations in the derivation of Lemma 3.

We consider the complementary event

$$E_1 : \forall S \subseteq [n] \text{ with } |S| = 2, U(S) \neq \emptyset.$$

Since $|S| = 2$, the event E_1 occurs if and only if for any two distinct rows $r_1 \neq r_2$, we have $\text{nz}(r_1) \neq \text{nz}(r_2)$. The probability of E_1 is:

$$\Pr[E_1] = \Pr[\text{Each row of } A \text{ is different}] = \prod_{i=0}^{n-1} \frac{M-i}{M}, \text{ where } M = \binom{\alpha n}{D}$$

The above equation comes from that each row of A has $\binom{\alpha n}{D}$ different choices.

We can derive the closed-form equation for the first term as:

$$\Pr \left[\exists S \subseteq [n], |S| = 2, U(S) = \emptyset \right] = 1 - \Pr[E_1] = 1 - \prod_{i=0}^{n-1} \left(1 - \frac{i}{\binom{\alpha n}{D}} \right) \quad (5)$$

The first term can be upper-bounded as:

$$\Pr \left[\exists S \subseteq [n], |S| = 2, U(S) = \emptyset \right] \leq \sum_{i=0}^{n-1} \frac{i}{\binom{\alpha n}{D}} = \frac{n(n-1)}{2^{\binom{\alpha n}{D}}} \quad (6)$$

For the other terms, we upper bound each of them using Lemma 3, and we have:

$$\sum_{s=3}^{\left\lfloor \frac{0.98n}{C} \right\rfloor} \Pr \left[\exists S \subseteq [n], |S| = s, U(S) = \emptyset \right] \leq \sum_{s=3}^{\left\lfloor \frac{0.98n}{C} \right\rfloor} F(s)$$

Based on the analysis in Lemma 3, $F(s)$ decreases on $[2, \frac{n}{C}]$ and increases on $[\frac{n}{C}, \frac{0.98n}{C}]$.

Furthermore, we observe that for small values of s , the function $F(s)$ decreases rapidly as s increases. Formally, we have the following lemma.

LEMMA 4. *If $s \leq \frac{n}{1.5eC} - 1$, then*

$$\frac{F(s+1)}{F(s)} \leq \frac{1}{1.5^{\frac{D}{2}-1}}.$$

PROOF. We compute

$$\frac{F(s+1)}{F(s)} = \frac{\left(\frac{C(s+1)}{n} \right)^{(s+1)(\frac{D}{2}-1)}}{\left(\frac{Cs}{n} \right)^{s(\frac{D}{2}-1)}} \\ = \left(\frac{s+1}{s} \right)^{s(\frac{D}{2}-1)} \cdot \left(\frac{C(s+1)}{n} \right)^{\frac{D}{2}-1}.$$

Since $(1 + \frac{1}{s})^s \leq e$ for all $s > 0$, it follows that

$$\frac{F(s+1)}{F(s)} \leq e^{\frac{D}{2}-1} \cdot \left(\frac{C(s+1)}{n} \right)^{\frac{D}{2}-1} = \left(\frac{eC(s+1)}{n} \right)^{\frac{D}{2}-1}.$$

Under the assumption $s + 1 \leq \frac{n}{1.5eC}$, we have $\frac{eC(s+1)}{n} \leq \frac{1}{1.5}$, and hence $\frac{F(s+1)}{F(s)} \leq \frac{1}{1.5^{\frac{D}{2}-1}}$ which finishes the proof. $\square$

By Lemma 4, for every integer s such that $3 \leq s \leq \lfloor \frac{n}{1.5eC} \rfloor - 1$, we have

$$\frac{F(s+1)}{F(s)} \leq \frac{1}{1.5^{\frac{D}{2}-1}}.$$

Iterating this inequality yields, for all $s \in \{3, \dots, \lfloor \frac{n}{1.5eC} \rfloor\}$,

$$F(s) \leq F(3) \cdot \frac{1}{1.5^{(\frac{D}{2}-1)(s-3)}}$$

Therefore,

$$\begin{aligned} \sum_{s=3}^{\lfloor \frac{n}{1.5eC} \rfloor} F(s) &\leq \sum_{s=3}^{\lfloor \frac{n}{1.5eC} \rfloor} F(3) \cdot \frac{1}{1.5^{(\frac{D}{2}-1)(s-3)}} \leq F(3) \sum_{t=0}^{\infty} \frac{1}{1.5^{(\frac{D}{2}-1)t}} \\ &= \frac{F(3)}{1 - \frac{1}{1.5^{(\frac{D}{2}-1)}}} \leq 1.8 F(3) \end{aligned} \quad (7)$$

, where in the last inequality we used $D \geq 6$, which implies $1 - \frac{1}{1.5^{\frac{D}{2}-1}} \geq \frac{5}{9}$.

Finally, we consider upper bounding $\sum_{s=\lfloor \frac{n}{1.5eC} \rfloor}^{\lfloor \frac{0.98n}{C} \rfloor} F(s)$.

LEMMA 5. For any integers $1 \leq a \leq b$ such that $\frac{Cb}{n} < 1$, we have

$$\sum_{s=a}^b \left(\frac{Cs}{n} \right)^{s(\frac{D}{2}-1)} \leq b \left(\frac{Cb}{n} \right)^{a(\frac{D}{2}-1)}.$$

PROOF. For all $s \in [a, b]$ we have $\frac{Cs}{n} \leq \frac{Cb}{n} < 1$, hence

$$\left(\frac{Cs}{n} \right)^{s(\frac{D}{2}-1)} \leq \left(\frac{Cb}{n} \right)^{s(\frac{D}{2}-1)}.$$

Moreover, since $\frac{Cb}{n} < 1$, the sequence $\left(\frac{Cb}{n} \right)^{s(\frac{D}{2}-1)}$ is decreasing in s , so for all $s \in [a, b]$,

$$\left(\frac{Cb}{n} \right)^{s(\frac{D}{2}-1)} \leq \left(\frac{Cb}{n} \right)^{a(\frac{D}{2}-1)}.$$

Therefore,

$$\begin{aligned} \sum_{s=a}^b \left(\frac{Cs}{n} \right)^{s(\frac{D}{2}-1)} &\leq \sum_{s=a}^b \left(\frac{Cb}{n} \right)^{s(\frac{D}{2}-1)} \leq (b-a+1) \left(\frac{Cb}{n} \right)^{a(\frac{D}{2}-1)} \\ &\leq b \left(\frac{Cb}{n} \right)^{a(\frac{D}{2}-1)} \end{aligned}$$

which proves Lemma 5. $\square$

To upper bound $\sum_{s=\lfloor \frac{n}{1.5eC} \rfloor}^{\lfloor \frac{0.98n}{C} \rfloor} F(s)$, let $a := \lfloor \frac{n}{1.5eC} \rfloor$ and $b := \lfloor \frac{0.98n}{C} \rfloor$. Since $\frac{Cb}{n} \leq 0.98 < 1$, Lemma 5 gives

$$\sum_{s=a}^b F(s) \leq b \left(\frac{Cb}{n} \right)^{a(\frac{D}{2}-1)} \leq \frac{n}{C} \cdot 0.98^{\lceil \frac{n}{1.5eC} \rceil (\frac{D}{2}-1)} \quad (8)$$

Combining Equation 6, Equation 7 and Equation 8, we have:

$$\begin{aligned} &\sum_{s=2}^{\lfloor \frac{0.98n}{C} \rfloor} \Pr \left[\exists S \subseteq [n], |S| = s, U(S) = \emptyset \right] \\ &\leq \frac{n(n-1)}{2 \binom{\alpha n}{D}} + 1.8 \left(\frac{3C}{n} \right)^{3(\frac{D}{2}-1)} + \frac{n}{C} \cdot 0.98^{\lceil \frac{n}{1.5eC} \rceil (\frac{D}{2}-1)} \end{aligned} \quad (9)$$

which proves our Theorem 8. $\square$

4.4 Cost Analysis and Determining Concrete Code Parameters

We instantiate Lightning by choosing the base code C_D to be the Brakedown code [18] with rate $\rho = 0.704$ and relative distance $\delta = 0.02$. We make this choice because Brakedown code is currently the only known field-agnostic linear code that admits linear encoding complexity. Under this configuration, encoding a message of length n using CD costs $9.9n$ field multiplications ($\mathbb{F}_\times$) and $9.9n$ field additions ($\mathbb{F}_+$).

The encoding cost of Lightning code constitutes two parts:

- (1) the sparse linear transformation $x \mapsto xA$ induced by the LDPC matrix A ;
- (2) the encoding of the base code C_D applied to the compressed input xA , which has length αn .

The first component computes xA and costs $Dn \mathbb{F}_+$, since each row of the matrix A contains exactly D nonzero entries, and all nonzero entries are equal to 1. Consequently, computing xA involves only field additions and no field multiplications.

The second component encodes a length- αn message using C_D , which costs $9.9\alpha n \mathbb{F}_\times$ and $9.9\alpha n \mathbb{F}_+$.

Combining the two components, the total encoding cost of Lightning on a length- n message is $9.9\alpha n \mathbb{F}_\times$ and $(9.9\alpha + D)n \mathbb{F}_+$.

The relative distance of the resulting Lightning code C_L is

$$\text{rel_dist}(C_L) = \min \left\{ \frac{\beta\rho}{\rho + \alpha}, \frac{\delta\alpha}{\rho + \alpha} \right\}.$$

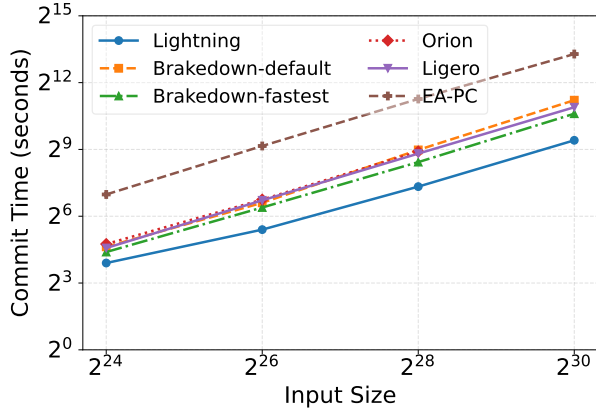
By Theorem 8, the parameter β is set to $\beta = \frac{0.98n}{C}$. Therefore, for any given pair (α, D) , the Lightning code is fully specified, and both its relative distance and failure probability can be computed explicitly using Theorem 8.

Although the failure probability depends on the message length n , we observe from Equation 9 that it is monotonically decreasing in n . As a result, it suffices to evaluate the failure probability at a moderate threshold value of n ; the same bound then holds for all larger message lengths. In this work, we focus on parameter choices suitable for large-scale PCS instances.

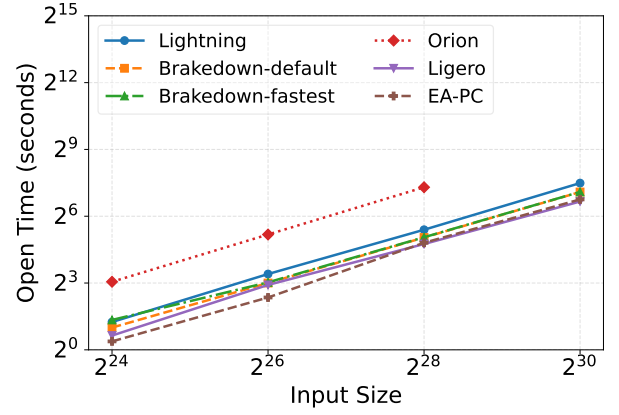
Finally, we perform a grid search over (α, D) to identify, for each target relative distance, the parameter set that minimizes the concrete encoding cost while ensuring a failure probability of at most 2^{-100} . Our setting of failure probability follows Brakedown [18] and EA [7].

4.5 Adding Zero-Knowledge

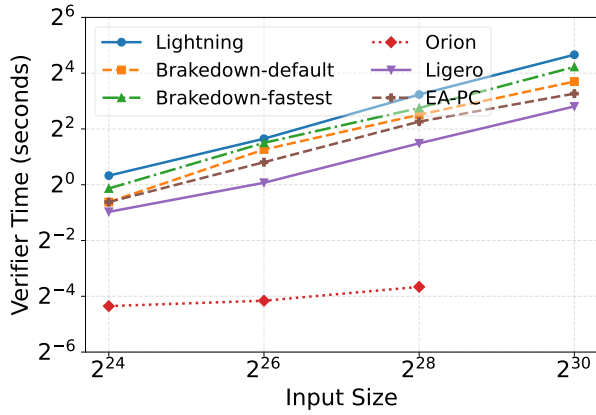
To make our PCS zero-knowledge, we apply the masking technique of [8, 33]. For each $i \in [k]$, the prover samples $R_i \leftarrow \mathbb{F}^n$ uniformly



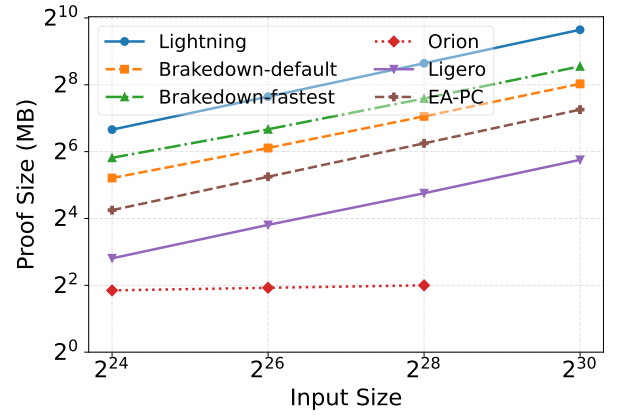
(a) Commit Time



(b) Open Time



(c) Verifier Time



(d) Proof Size

Figure 2: Benchmark results. Lightning uses the parameters in Table 3 with relative distance 0.004. Brakedown-default follows the default settings in [18], using the Brakedown parameters with distance=0.04. Brakedown-fastest uses the optimal cost parameters in Table 2. Ligerio [3] uses Reed-Solomon code rate= $\frac{1}{2}$. Orion and EA use their default parameters in their paper.

at random and replaces the i -th row's codeword $E_C(W[i])$ with the randomized encoding $(E_C(W[i]) + R_i \parallel R_i)$ of length $2n$, where $\parallel$ denotes concatenation. By construction, for any column index $j \in [n]$, the value at position j is a uniformly random field element as long as the value at position $j + n$ remains hidden (and vice versa). Accordingly, the verifier samples an opening set $\hat{I} \subset [2n]$ with $|\hat{I}| = 2t$, consisting of t indices from $[n]$ and t indices from $[n + 1, 2n]$, such that for every $i \in [n]$, $\hat{I}$ contains at most one of i and $i + n$.

5 Experiments

In this section, we present a comprehensive evaluation of our Lightning PCS.

Software. Following [7, 18], we use a 254-bit prime in our experiments. We develop our PCS in C++ language, implementing the field operations using the mcl library¹.

¹<https://github.com/herumi/mcl>

Hardware. All the experiments are conducted on a cloud server equipped with 90 GB memory and 16 Intel CPU cores. We always use a single thread to execute the experiments.

Baselines. In this paper, we compare the performance of our PCS against existing coding-based PCS, including Brakedown [18] (CRYPTO'23), Orion [33] (CRYPTO'22), Ligerio [3] (CCS'17), and EA [7] (CRYPTO'24). For Brakedown, Orion, and EA, we adopt their official implementation. For Ligerio, we rely on the Ligerio implementation included in the Brakedown codebase [18].

Hyperparameters. By default, we use the Lightning code with code rate 0.720 and relative distance 0.004 in Table 3. All commitment schemes are configured to achieve 128-bit computational soundness. The failure probabilities of Lightning, Brakedown, Orion, and EA are each set to be at most 2^{-100} .

5.1 Performance for Lightning PCS

Figure 2 presents the experimental results. Orion [33] exhausted available memory when processing polynomials of degree 2^{30} .

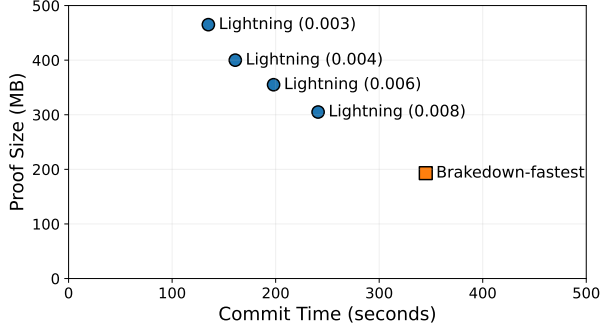


Figure 3: Commit time–proof size trade-off for different Lightning configurations (with relative distances shown in parentheses), compared to Brakedown-fastest (relative distance 0.016).

As shown in Figure 2, for sufficiently large polynomials, Lightning achieves a significantly faster commitment time than existing coding-based PCS: roughly 2.0–2.3× faster than Brakedown-fastest, 2.5–3.0× faster than Orion, and 14–15× faster than EA-PC. In all existing coding-based PCS, the prover’s dominant cost is committing to the polynomial, while the opening time is substantially shorter.

In terms of communication, the fastest baseline, Brakedown-fastest, yields proofs 2.0–2.2× smaller than Lightning. All baselines except Orion have proof sizes of $O(\sqrt{N})$, whereas Orion achieves the best asymptotics with $O(\log^2 N)$ proof size and also the smallest concrete proof size.

In summary, Lightning offers a prover that is both concretely and asymptotically fast, but it produces larger proofs and slower verification compared to other schemes. The larger proof size stems from the smaller relative distance of its underlying code, which requires more column openings. Although Orion and Ligero achieve smaller proof sizes among coding-based PCS, neither is field-agnostic—unlike Lightning, which maintains field-agnostic.

5.2 Commit Time–Proof Size Tradeoff Across Code Distances

As shown in Table 3, different Lightning code configurations achieve different tradeoffs between encoding cost and relative distance. This results in different Lightning PCS configurations with different commit time–proof size tradeoffs.

To explore the resulting prover time–proof size tradeoff, we instantiate Lightning PCS with four configurations at relative distances 0.003, 0.004, 0.006, and 0.008 in Table 3, and we compare them against Brakedown-fastest (relative distance 0.016). All schemes are evaluated on multilinear polynomials of size 2^{28} .

Figure 3 plots the commit time against proof size for each configuration. As the relative distance of Lightning decreases from 0.008 to 0.003, the commit time drops from 241s to 135s, while the proof size grows from 305MB to 465MB. Brakedown-fastest, on the other hand, requires 345s to commit but produces a proof of only 193MB. The two schemes span a Pareto frontier, with Lightning occupying the fast-prover/large-proof end and Brakedown occupying the slow-prover/small-proof end.

This tradeoff can be explained by the encoding cost structure of the two schemes. As shown in Table 2, when the target relative distance is small (e.g., $\delta \leq 0.01$), Lightning achieves lower encoding cost than even the optimal-cost Brakedown configuration: at $\delta = 0.004$, Lightning requires only $5.6n$ equivalent field multiplications compared to Brakedown’s $12.3n$, a reduction of more than 2×. However, the smaller relative distance of Lightning yields a proof size larger than that of Brakedown.

6 Suitable Application Scenarios

Lightning is particularly well-suited for application scenarios where prover efficiency is the primary bottleneck, and communication cost is comparatively less critical. A prominent example is zero-knowledge machine learning (zkML) [22, 27, 28, 31]. Unlike blockchain-based deployments, where every additional byte in the proof directly translates to monetary cost (e.g., gas fees), zkML workloads are typically executed in off-chain settings, where proofs are transmitted over standard network channels [27]. In such environments, the dominant constraint is the prover’s computational cost, while moderately larger proofs—on the order of hundreds of megabytes—remain acceptable given modern network bandwidth. For example, a 500 MB proof can be transmitted in under a minute over commodity broadband connections, making communication overhead secondary to computation.

In zkML workloads, prover cost dominates overall system efficiency. Existing systems already make design choices that prioritize prover efficiency over communication cost. For example, zkPoT [1] (CCS’24) adopts Orion [33] as its PCS, which incurs relatively large proof sizes compared to alternatives such as KZG [20], PST [26], Hyrax [32], and FRI [6].

In these settings, Lightning directly targets this bottleneck by reducing prover commitment cost. In our evaluation, it achieves up to 2× lower cost than Brakedown and approximately 3× improvement over Orion. While this comes at the expense of increased proof size, the resulting proofs remain within the practical range already tolerated by such systems, making the tradeoff favorable for off-chain deployments.

More broadly, Lightning is advantageous in any setting characterized by (i) large input sizes requiring expensive polynomial encodings, (ii) limited prover-side computational resources, or (iii) latency-sensitive proving pipelines. In contrast, in settings where communication cost dominates—such as on-chain verification—schemes with smaller proof size (e.g., KZG) may be preferable.

7 Conclusion

We present *Lightning*, a new coding-based PCS that significantly improves prover efficiency. By introducing the Lightning code, we reduce encoding cost while preserving constant relative distance. This improvement directly translates to an intriguing trade-off between prover time and proof size in the resulting PCS. Experimental results demonstrate that Lightning PCS achieves state-of-the-art commitment efficiency among coding-based PCS, while incurring only a moderate increase in proof size. These results establish Lightning as an effective approach for improving prover efficiency in zkSNARK systems.

References

- [1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. 2024. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4316–4330.
- [2] Martin R. Albrecht, Giacomo Fenzi, Oleksandra Lapiha, and Ngoc Khanh Nguyen. 2024. SLAP: Succinct Lattice-Based Polynomial Commitments from Standard Assumptions. In *Advances in Cryptology – EUROCRYPT 2024*, Marc Joye and Gregor Leander (Eds.). Springer Nature Switzerland, Cham, 90–119.
- [3] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. 2017. Liger: Lightweight sublinear arguments without a trusted setup. In *Proceedings of the 2017 acm sigsac conference on computer and communications security*. 2087–2104.
- [4] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. 2025. WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification. In *Advances in Cryptology – EUROCRYPT 2025*, Serge Fehr and Pierre-Alain Fouque (Eds.). Springer Nature Switzerland, Cham, 214–243.
- [5] Arasu Arun, Srinath Setty, and Justin Thaler. 2024. Jolt: SNARKs for Virtual Machines via Lookups. In *Advances in Cryptology – EUROCRYPT 2024: 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zurich, Switzerland, May 26–30, 2024, *Proceedings, Part VI*.
- [6] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *Advances in Cryptology – EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Darmstadt, Germany, May 19–23, 2019, *Proceedings, Part I* (Darmstadt, Germany). Springer-Verlag, Berlin, Heidelberg, 103–128. doi:10.1007/978-3-030-17653-2_4
- [7] Alexander R Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. 2024. Field-agnostic SNARKs from expand-accumulate codes. In *Annual International Cryptology Conference*. Springer, 276–307.
- [8] Jonathan Bootle, Andrea Cerulli, Essam Ghadafi, Jens Groth, Mohammad Hajiabadi, and Sune K. Jakobsen. 2017. Linear-Time Zero-Knowledge Proofs for Arithmetic Circuit Satisfiability. In *Advances in Cryptology – ASIACRYPT 2017*.
- [9] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. 2020. Linear-time arguments with sublinear verification from tensor codes. In *Theory of Cryptography Conference*. Springer, 19–46.
- [10] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. 2022. Correlated pseudorandomness from expand-accumulate codes. In *Annual International Cryptology Conference*. Springer, 603–633.
- [11] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. 2025. Blaze: Fast SNARKs from Interleaved RAA Codes. In *Advances in Cryptology – EUROCRYPT 2025: 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Madrid, Spain, May 4–8, 2025, *Proceedings, Part IV* (Madrid, Spain). Springer-Verlag, Berlin, Heidelberg, 123–152. doi:10.1007/978-3-031-91134-7_5
- [12] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. 2020. Transparent SNARKs from DARK Compilers. In *Advances in Cryptology – EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zagreb, Croatia, May 10–14, 2020, *Proceedings, Part I* (Zagreb, Croatia). Springer-Verlag, Berlin, Heidelberg, 677–706. doi:10.1007/978-3-030-45721-1_24
- [13] Miranda Christ and Sam Gunn. 2024. Pseudorandom Error-Correcting Codes. In *Advances in Cryptology – CRYPTO 2024*, Leonid Reyzin and Douglas Stebila (Eds.). Springer Nature Switzerland, Cham, 325–347.
- [14] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. 2012. Practical verified computation with streaming interactive proofs. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. 90–112.
- [15] Geoffroy Couteau, Peter Rindal, and Srinivasan Raghuraman. 2021. Silver: silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes. In *Annual International Cryptology Conference*. Springer, 502–534.
- [16] Dariush Divsalar, Hui Jin, and Robert J McEliece. 1998. Coding theorems for "turbo-like" codes. In *Proceedings of the annual Allerton Conference on Communication control and Computing*, Vol. 36. University Of Illinois, 201–210.
- [17] R. Gallager. 1962. Low-density parity-check codes. *IRE Transactions on Information Theory* 8, 1 (1962), 21–28. doi:10.1109/TIT.1962.1057683
- [18] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS. In *Advances in Cryptology – CRYPTO 2023*.
- [19] Ulrich Haböck. 2023. Brakedown's expander code. *Cryptology ePrint Archive* (2023).
- [20] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 5-9, 2010. *Proceedings 16*. Springer, 177–194.
- [21] Jonathan Lee. 2021. Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments. In *Theory of Cryptography*, Kobbi Nissim and Brent Waters (Eds.). Springer International Publishing, Cham, 1–34.
- [22] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. ZkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2968–2985.
- [23] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. 2025. Ceno: Non-uniform, Segment and Parallel Zero-Knowledge Virtual Machine. *J. Cryptol.* 38, 2 (Jan. 2025), 56 pages. doi:10.1007/s00145-024-09533-2
- [24] Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. 2021. CanDID: Can-Do Decentralized Identity with Legacy Compatibility, Sybil-Resistance, and Accountability. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1348–1366. doi:10.1109/SP40001.2021.00038
- [25] Vineet Nair, Ashish Sharma, and Bhargav Thankey. 2026. BrakingBase - A Linear Prover, Poly-Logarithmic Verifier, Field Agnostic Polynomial Commitment Scheme. In *Advances in Cryptology – ASIACRYPT 2025*, Goichiro Hanaoka and Bo-Yin Yang (Eds.). Springer Nature Singapore, Singapore, 427–455.
- [26] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. 2013. Signatures of correct computation. In *Theory of Cryptography Conference*. Springer, 222–242.
- [27] Wenjie Qu, Yanpei Guo, Yue Ying, and Jiaheng Zhang. 2025. VerfCNN, Optimal Complexity zkSNARK for Convolutional Neural Networks. *Cryptology ePrint Archive* (2025).
- [28] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium (USENIX Security 25)*.
- [29] Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE symposium on security and privacy*. IEEE, 459–474.
- [30] Daniel A Spielman. 1995. Linear-time encodable and decodable error-correcting codes. In *Proceedings of the twenty-seventh annual ACM symposium on Theory of computing*. 388–397.
- [31] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. *arXiv preprint arXiv:2404.16109* (2024).
- [32] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. 2018. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 926–943.
- [33] Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2022. Orion: Zero knowledge proof with linear prover time. In *Annual International Cryptology Conference*. Springer, 299–328.
- [34] Hadas Zeilberger, Binyi Chen, and Ben Fisch. 2024. BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *Annual International Cryptology Conference*. Springer, 138–169.
- [35] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2020. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 859–876.

A Open Science

The anonymous <https://anonymous.4open.science/r/Lightning-31D2> repository includes our system's implementation. All artifacts are accessible at the time of submission, in accordance with the CCS open science policy.

B Ethical Considerations

This work studies polynomial commitment schemes, a cryptographic primitive used in zkSNARKs. It does not involve human subjects, user data, or real-world deployment. PCS enables verifiable and privacy-preserving computation, and our contribution improves prover efficiency. While cryptographic techniques are inherently dual-use, our work does not introduce new capabilities beyond existing frameworks.

C Acknowledgements

This paper was edited for grammar using GPT-5.